

Simple Pairs Trading Strategy Using Cointegration

Code Explanation & Technical Guide

`pairs_trading.py`

1. Project Overview

This project implements a statistical arbitrage strategy on S&P 500 equities. The core idea is to find pairs of stocks whose prices move together over the long run (cointegration), and to trade the temporary deviations between them expecting mean reversion.

The script is a complete, self-contained pipeline that handles every step from raw data download through to out-of-sample performance reporting.

High-level flow

Step	Function	Purpose
1	get_sp500_tickers()	Fetch ticker list from Wikipedia
2	download_prices()	Download adjusted close prices via yfinance
3	test_cointegration()	Engle-Granger test on all stock pairs
4	compute_spread_zscore()	Rolling OLS hedge ratio, spread, z-score
5	optimize_parameters()	Grid-search over entry/exit thresholds
6	backtest_pair()	Simulate long/short trades with costs
7	performance_metrics()	Sharpe, drawdown, win rate, etc.
8	Plotting functions	Save all charts to outputs/

2. Configuration Block

All key parameters are defined at the top of the file for easy adjustment without touching the logic:

```

INSAMPLE_START   = '2018-01-01'   # Training period start
INSAMPLE_END     = '2021-12-31'   # Training period end
OUTSAMPLE_START  = '2022-01-01'   # Test period start
OUTSAMPLE_END    = '2023-12-31'   # Test period end

COINT_PVAL       = 0.05           # Max p-value to accept a pair
ENTRY_ZSCORE     = 2.0            # Open trade when |z| exceeds this
EXIT_ZSCORE      = 0.0            # Close trade when |z| drops below this
STOP_LOSS_ZSCORE = 3.5            # Force-close if spread diverges further
MAX_HOLD_DAYS    = 60             # Prevent stale positions
TRANSACTION_COST = 0.001          # 10 bps per side
LOOKBACK         = 60             # Rolling window for hedge ratio & z-score

```

The in-sample period is used to find pairs and optimise thresholds. The out-of-sample period is never touched during parameter selection - this is the key anti-overfitting discipline.

3. Data Download

get_sp500_tickers(n)

Reads the S&P 500 constituent table from Wikipedia using `pd.read_html()`. Dots in tickers (e.g. BRK.B) are replaced with hyphens to match Yahoo Finance format. A hardcoded fallback of 50 liquid tickers is used if the

HTTP request fails.

download_prices(tickers, start, end)

Uses `yf.download()` in batch mode to retrieve daily adjusted close prices for all tickers in a single API call. Two cleaning steps are applied:

- Columns with more than 10% missing values are dropped entirely.
- Remaining gaps are forward-filled, then any remaining NaN rows are dropped.

Only the 'Close' (auto-adjusted for splits and dividends) field is retained. The result is a clean T x N DataFrame of prices.

4. Cointegration Testing

Why cointegration, not correlation?

Two stocks can be highly correlated over a period yet still drift apart permanently. Cointegration is a stronger statistical concept: it means there exists a linear combination of the two price series that is stationary (mean-reverting). This is the prerequisite for a pairs trade - without it, the spread may never return to its historical mean.

test_cointegration(prices, pval_thresh)

Iterates over every unique stock pair using `itertools.combinations`. For each pair (S1, S2) it calls `statsmodels.tsa.stattools.coint()`, which runs the Engle-Granger two-step test:

- Step 1: Regress S1 on S2 and save the residuals (the spread).
- Step 2: Run an ADF (Augmented Dickey-Fuller) test on the residuals.
- If the ADF p-value is below the threshold, the pair is cointegrated.

Pairs that pass the threshold are returned sorted by p-value (most significant first). The Pearson correlation between each pair is also stored for reference - high correlation does NOT guarantee cointegration.

plot_pvalue_heatmap()

Builds a symmetric N x N matrix of p-values and renders it as a color heatmap (green = significant, red = not). Non-tested entries are filled with 1.0. Saved to `outputs/coint_heatmap.png`.

5. Spread Construction & Z-score Signal

compute_hedge_ratio(y, x)

Runs OLS regression of stock Y on stock X (with intercept). The slope coefficient is the hedge ratio beta. The spread is defined as: $\text{spread} = Y - \text{beta} * X$

compute_spread_zscore(y, x, lookback)

Uses a rolling window approach so the hedge ratio and z-score adapt over time, avoiding look-ahead bias:

```
for i in range(lookback, len(y)):
    window_y = y[i-lookback : i]      # past 'lookback' days only
    window_x = x[i-lookback : i]
    beta      = OLS(window_y, window_x).fit().params[1]
    spread[i] = y[i] - beta * x[i]

zscore = (spread - rolling_mean) / rolling_std
```

The z-score tells us how many standard deviations the current spread is from its recent average. A z-score of +2 means the spread is 2 standard deviations above its mean - suggesting it will revert downward.

ADF verification

`adf_test()` runs an ADF test on the computed spread to confirm stationarity. If the spread is not stationary out

of sample, the cointegration relationship has broken down and the pair should be discarded.

6. Backtesting Engine

Trading logic - `backtest_pair()`

The backtester iterates day-by-day through the z-score series:

Condition	Action
$z < -\text{entry}$	Long spread: buy Y, short $\beta \cdot X$
$z > +\text{entry}$	Short spread: sell Y, buy $\beta \cdot X$
$ z < \text{exit (long)}$	Close long position - mean reverted
$ z < \text{exit (short)}$	Close short position - mean reverted
$ z > \text{stop_loss}$	Stop-loss: close regardless of direction
$\text{hold_days} \geq \text{max_hold}$	Time-stop: close stale position

Transaction costs

A flat cost of `TRANSACTION_COST` (default 0.001 = 10 bps) is subtracted from the P&L on every entry and exit. This models bid-ask spread and broker commissions. The P&L on each day is the position (+1 or -1) multiplied by the change in spread.

Output

`backtest_pair()` returns two objects: (1) a daily P&L DataFrame with a cumulative return column, and (2) a trade log recording entry date, direction, hold days, total return, and exit reason.

7. Parameter Optimisation

`optimize_parameters()`

Runs a grid search over entry and exit z-score thresholds on in-sample data only. For each combination the Sharpe ratio is computed. The best parameter set is then used for the out-of-sample test.

```
entry_range = [1.5, 2.0, 2.5]
exit_range = [0.0, 0.25, 0.5]
# 9 combinations evaluated per pair
```

The sensitivity is also visualised as a 3x3 heatmap (`outputs/sensitivity_S1_S2.png`), making it easy to see whether performance is robust or fragile to threshold changes.

Important: parameter optimisation happens entirely within the in-sample window. The out-of-sample period is used only for final evaluation, preventing data snooping.

8. Performance Metrics

Metric	Formula	Interpretation
Annualised Return	$\text{daily_mean} * 252$	Scales daily P&L to yearly
Annualised Vol	$\text{daily_std} * \sqrt{252}$	Scales daily volatility to yearly
Sharpe Ratio	$\text{ann_return} / \text{ann_vol}$	Risk-adjusted return; higher is better

Pairs Trading Strategy - Code Explanation		
Max Drawdown	<code>min(cumret - running_max(cumret))</code>	Worst peak-to-trough loss
Win Rate	<code>trades_with_positive_return / n_trades</code>	Fraction of profitable trades
Avg Trade Return	<code>mean(trade_returns)</code>	Average P&L per trade
N Trades	<code>len(trade_log)</code>	Total number of completed trades

9. Output Files

File	Description
coint_heatmap.png	Heatmap of cointegration p-values across all pairs
cointegration_pvalues.csv	Table of all significant pairs with p-values and correlation
spread_S1_S2.png	Spread and z-score chart with entry/exit bands (per pair)
sensitivity_S1_S2.png	3x3 Sharpe heatmap across entry/exit thresholds (per pair)
equity_curve_IS.png	Cumulative P&L for each pair + portfolio (in-sample)
equity_curve_OOS.png	Cumulative P&L for each pair + portfolio (out-of-sample)
trade_log_S1_S2_IS.csv	Detailed log of every trade (per pair)
performance_summary.csv	In-sample vs out-of-sample metrics for all pairs

10. How to Run

Install dependencies

```
pip install yfinance statsmodels pandas numpy matplotlib seaborn scipy scikit-learn
```

Run the script

```
python pairs_trading.py
```

The script will automatically download data, run all tests, and save outputs/ in the current directory. No manual data download is needed.

Adjusting the stock universe

Change `n=50` in `get_sp500_tickers(n=50)` to scan more pairs (note: runtime grows as $O(n^2)$ due to pairwise cointegration testing).

Key things to investigate

- Do in-sample cointegrated pairs remain cointegrated out of sample?
- Does high correlation actually imply cointegration?
- How much does performance degrade after transaction costs?
- Is profit driven by one or two large trades, or is it consistent?
- How sensitive is the Sharpe ratio to the entry/exit thresholds?